

# Python Conditionals & Loops

if / elif / else · for / while · break / continue / pass · Comprehensions

## 1. COMPARISON & LOGICAL OPERATORS

These are the building blocks of every condition. You must know these before writing any if/else.

### ► Comparison Operators — Return True or False

| Operator               | Meaning & Example                                         |
|------------------------|-----------------------------------------------------------|
| <code>x == y</code>    | Equal to → <code>5 == 5</code> is True                    |
| <code>x != y</code>    | Not equal to → <code>5 != 3</code> is True                |
| <code>x &gt; y</code>  | Greater than → <code>7 &gt; 3</code> is True              |
| <code>x &lt; y</code>  | Less than → <code>2 &lt; 9</code> is True                 |
| <code>x &gt;= y</code> | Greater than or equal to → <code>5 &gt;= 5</code> is True |
| <code>x &lt;= y</code> | Less than or equal to → <code>4 &lt;= 6</code> is True    |

### ► Logical Operators — Combine Conditions

| Operator | Rule & Example                                        |
|----------|-------------------------------------------------------|
| and      | Both conditions must be True → True and False → False |
| or       | At least one must be True → True or False → True      |
| not      | Flips True/False → not True → False                   |

### ► Identity & Membership Operators

| Operator                | What It Checks                                    |
|-------------------------|---------------------------------------------------|
| <code>x is y</code>     | True if x and y point to SAME object in memory    |
| <code>x is not y</code> | True if x and y are different objects             |
| <code>x in y</code>     | True if x exists inside y (string, list, dict...) |
| <code>x not in y</code> | True if x does NOT exist inside y                 |

#### EXAMPLE

```
a = [1, 2, 3]
b = a          # b points to SAME list
c = [1, 2, 3]  # c is a DIFFERENT list with same values
```

```

print(b is a)          # True - same object
print(c is a)          # False - different objects
print(c == a)          # True - same values

print(2 in a)          # True
print(5 not in a)      # True
print("ello" in "Hello") # True

```

OUT

```

True
False
True
True
True
True

```

❓ Use == to compare VALUES, use is to compare IDENTITY (same object). Never use 'is' to compare integers/strings in general.

## ❓ 2. IF / ELIF / ELSE — CONDITIONALS

### ► Basic if / elif / else

#### SYNTAX

```

# SYNTAX
if condition1:
    # runs if condition1 is True
elif condition2:
    # runs if condition1 False AND condition2 True
else:
    # runs if ALL above conditions are False

```

#### EXAMPLE

```

marks = 72

if marks >= 90:
    grade = 'A+'
elif marks >= 80:
    grade = 'A'
elif marks >= 70:
    grade = 'B'
elif marks >= 60:
    grade = 'C'
else:
    grade = 'Fail'

print(f'Marks: {marks} → Grade: {grade}')

```

OUT

Marks: 72 → Grade: B

❓ Python evaluates conditions top-to-bottom and STOPS at the first True. Only one block runs.

## ► Nested if (if inside if)

### EXAMPLE

```
age = 20
has_id = True

if age >= 18:
    if has_id:          # nested condition
        print('Entry allowed')
    else:
        print('No ID – Entry denied')
else:
    print('Under 18 – Entry denied')
```

OUT

Entry allowed

## ► Ternary / Inline if (One-liner)

Write a simple if/else on a single line. Best for simple assignments.

### EXAMPLE

```
# value_if_true if condition else value_if_false
age = 20
status = 'Adult' if age >= 18 else 'Minor'
print(status)  # Adult

# Works in f-strings too
n = -5
print(f'{n} is {"positive" if n > 0 else "non-positive"}')
```

OUT

Adult  
-5 is non-positive

## ► Truthy and Falsy Values

In Python, conditions don't have to be True/False literally — any value is evaluated as truthy or falsy.

| Category      | Examples                                                                                       |
|---------------|------------------------------------------------------------------------------------------------|
| Falsy values  | False, None, 0, 0.0, "" (empty str), [] (empty list), {} (empty dict), () (empty tuple), set() |
| Truthy values | Everything else — non-zero numbers, non-empty strings, non-empty collections                   |

### TRUTHY / FALSY DEMO

```
values = [0, '', [], None, False, 42, 'hello', [1,2]]

for v in values:
    if v:          # truthy check
        print(f'{repr(v):12} → Truthy')
    else:
        print(f'{repr(v):12} → Falsy')
```

OUT

0  
' '  
[]  
None  
False  
42  
'hello'  
[1, 2]

→ Falsy  
→ Falsy  
→ Falsy  
→ Falsy  
→ Falsy  
→ Truthy  
→ Truthy  
→ Truthy

### 3. MATCH-CASE — STRUCTURAL PATTERN MATCHING (Python 3.10+)

match-case is Python's modern switch statement. It matches a value against patterns — cleaner than long if/elif chains.

#### SYNTAX

```
# SYNTAX
match subject:
    case pattern1:
        # action
    case pattern2:
        # action
    case _:          # wildcard – matches anything (like 'else')
        # default action
```

#### EXAMPLE

```
def http_status(code):
    match code:
        case 200: return 'OK'
        case 404: return 'Not Found'
        case 500: return 'Server Error'
        case 301 | 302: return 'Redirect'    # OR patterns
        case _: return 'Unknown'

print(http_status(200))    # OK
print(http_status(404))    # Not Found
print(http_status(302))    # Redirect
print(http_status(999))    # Unknown
```

#### OUT

```
OK
Not Found
Redirect
Unknown
```

match-case is NOT the same as C/Java switch. It does structural pattern matching — can match types, tuples, lists, and even guard conditions (if clauses).

### 4. FOR LOOP

Iterates over any iterable — list, string, range, tuple, dict, set, file, generator...

#### SYNTAX

```
for variable in iterable:
    # body runs once per item
```

#### ► 4a. Looping over Lists, Strings, Tuples

#### EXAMPLE

```
fruits = ['apple', 'banana', 'cherry']
```

```
for fruit in fruits:
    print(fruit)

# Strings are iterable – char by char
for ch in "Hi!":
    print(ch, end=' ')
```

OUT

```
apple
banana
cherry
H i !
```

#### ► 4b. range() — Generate Number Sequences

| range() Form             | Produces                                      |
|--------------------------|-----------------------------------------------|
| range(stop)              | 0 to stop-1 → range(5) → 0,1,2,3,4            |
| range(start, stop)       | start to stop-1 → range(2,6) → 2,3,4,5        |
| range(start, stop, step) | step controls gap → range(0,10,2) → 0,2,4,6,8 |
| range(10, 0, -1)         | count backwards → 10,9,8...1                  |

#### EXAMPLE

```
# Sum of 1 to 10
total = 0
for i in range(1, 11):    # 1,2,3,...,10
    total += i
print(total)    # 55

# Countdown
for i in range(5, 0, -1):
    print(i, end=' ')    # 5 4 3 2 1
```

OUT

```
55
5 4 3 2 1
```

#### ► 4c. enumerate() — Index + Value Together

Get both the index and the value without using range(len(...)). Clean and Pythonic.

#### EXAMPLE

```
languages = ['Python', 'Java', 'C++', 'Go']

for index, lang in enumerate(languages):
    print(f'{index}: {lang}')

# Start index from 1
for i, lang in enumerate(languages, start=1):
    print(f'{i}. {lang}')
```

OUT

```
0: Python
1: Java
2: C++
```

```
3: Go
1. Python
2. Java
3. C++
4. Go
```

#### ► 4d. zip() — Loop over Multiple Lists Together

##### EXAMPLE

```
names = ['Aman', 'Priya', 'Rohit']
scores = [85, 92, 78]
cities = ['Delhi', 'Mumbai', 'Chennai']

for name, score, city in zip(names, scores, cities):
    print(f'{name} ({city}): {score}')

# zip stops at shortest list
# Use zip_longest from itertools to fill missing values
```

OUT

```
Aman (Delhi): 85
Priya (Mumbai): 92
Rohit (Chennai): 78
```

#### ► 4e. Looping over Dictionaries

##### DICT LOOPING

```
student = {'name': 'Nishant', 'age': 25, 'city': 'Delhi'}

# Keys only (default)
for key in student:
    print(key)

# Values only
for val in student.values():
    print(val)

# Keys AND Values
for key, val in student.items():
    print(f'{key}: {val}')
```

OUT

```
name age city
---
Nishant 25 Delhi
---
name: Nishant
age: 25
city: Delhi
```

## 5. WHILE LOOP

Runs as long as a condition is True. Use when you DON'T KNOW how many iterations are needed.

### SYNTAX

```
while condition:
    # body – runs repeatedly until condition is False
```

### ► Basic while Loop

#### EXAMPLE

```
count = 1
while count <= 5:
    print(count, end=' ')
    count += 1          # MUST update, or infinite loop!

print() # newline
print('Done!')
```

OUT

1 2 3 4 5  
Done!

Always update the loop variable inside while — otherwise you get an infinite loop!

### ► while with User Input (Real-world pattern)

#### EXAMPLE — Input Validation

```
# Keep asking until valid input
while True:
    age = input('Enter your age: ')
    if age.isdigit() and int(age) > 0:
        print(f'Valid age: {age}')
        break          # exit when valid
    print('Invalid! Try again.')
```

### ► while with Counter / Accumulator

#### REAL-WORLD EXAMPLE

```
# Guess the number game
import random
secret = random.randint(1, 10)
attempts = 0

while True:
    guess = int(input('Guess (1-10): '))
    attempts += 1
    if guess == secret:
        print(f'Correct! Took {attempts} attempts.')
        break
    elif guess < secret:
        print('Too low!')
    else:
```



```
print('Too high!')
```

### ► for vs while — When to use which?

| Aspect        | for Loop                        | while Loop                  |
|---------------|---------------------------------|-----------------------------|
| When to use   | Know the number of iterations   | Don't know when it ends     |
| Iterates over | Any iterable (list, range, str) | A condition                 |
| Risk          | No infinite loop risk           | Easy to get infinite loop   |
| Common use    | Lists, ranges, dicts            | User input, polling, games  |
| Example       | for i in range(10):             | while user_input != 'quit': |

## 6. BREAK, CONTINUE, PASS — LOOP CONTROL

### ► break — Exit the loop immediately

Stops the loop the moment it executes. Jumps to the code after the loop.

#### EXAMPLE

```
numbers = [3, 7, 2, 9, 4, 1, 8]

for num in numbers:
    if num == 9:
        print(f'Found 9! Stopping.')
        break          # exits for loop here
    print(num)

print('After loop')
```

OUT

```
3
7
2
Found 9! Stopping.
After loop
```

### ► continue — Skip current iteration, keep going

Skips the rest of the current iteration's body and jumps back to the next iteration.

#### EXAMPLE

```
for i in range(1, 11):
    if i % 2 == 0:      # skip even numbers
        continue
    print(i, end=' ')  # only odd numbers printed

# Real use: skip invalid data
data = [10, -1, 25, None, 30, -5, 40]
total = 0
```

```

for val in data:
    if val is None or val < 0:
        continue      # skip bad values
    total += val
print(f'Total: {total}')

```

OUT

```

1 3 5 7 9
Total: 105

```

## ► pass — Do nothing (placeholder)

A no-op statement. Used as a placeholder where Python syntax requires a body but you have nothing to write yet.

### EXAMPLE

```

# Placeholder for future code
def process_data():
    pass                # TODO: implement later

class EmptyClass:
    pass                # valid empty class

# In loops – skip certain cases silently
for i in range(10):
    if i % 3 == 0:
        pass            # handle multiples of 3 later
    else:
        print(i, end=' ')

```

OUT

```

1 2 4 5 7 8

```

## ► else with Loops — The 'no break' clause

A loop's else block runs ONLY if the loop completed normally (no break). If break was hit, else is skipped.

### EXAMPLE — for/else

```

# Search for a prime number
def is_prime(n):
    if n < 2: return False
    for i in range(2, int(n**0.5)+1):
        if n % i == 0:
            print(f'{n} is NOT prime (divisible by {i})')
            break        # found a divisor
    else:
        # Only runs if no break occurred
        print(f'{n} IS prime!')
        return True
    return False

is_prime(7)
is_prime(9)
is_prime(13)

```

OUT

```

7 IS prime!
9 is NOT prime (divisible by 3)
13 IS prime!

```

❓ Loop else is rarely seen but very Pythonic. Perfect for search algorithms — 'if the loop ended without break, item was not found'.

#### ► break / continue / pass — Side-by-Side

| Keyword         | Effect                          | Continues loop?              |
|-----------------|---------------------------------|------------------------------|
| <b>break</b>    | Exits the loop entirely         | No — jumps after loop        |
| <b>continue</b> | Skips rest of current iteration | Yes — goes to next iteration |
| <b>pass</b>     | Does nothing at all             | Yes — continues normally     |

## 7. NESTED LOOPS

A loop inside another loop. Inner loop runs completely for each iteration of the outer loop.

### ► Multiplication Table

#### EXAMPLE

```
for i in range(1, 4):          # outer - rows
    for j in range(1, 4):      # inner - columns
        print(f'{i}x{j}={i*j}', end=' ')
    print()                   # newline after each row
```

OUT

```
1x1=1  1x2=2  1x3=3
2x1=2  2x2=4  2x3=6
3x1=3  3x2=6  3x3=9
```

### ► Pattern Printing (Classic Interview Questions)

#### EXAMPLE

```
# Right triangle pattern
n = 5
for i in range(1, n+1):
    print('* ' * i)

# Number triangle
for i in range(1, n+1):
    for j in range(1, i+1):
        print(j, end=' ')
    print()
```

OUT

```
*
* *
* * *
* * * *
* * * * *

---
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

🔖 break inside a nested loop only breaks the INNERMOST loop. Use a flag variable or refactor into a function to break out of multiple loops.

### ► Breaking Out of Nested Loops — Flag Pattern

#### BREAK OUT OF NESTED

```
found = False
matrix = [[1,2,3],[4,5,6],[7,8,9]]
target = 5

for row in matrix:
    for val in row:
```

```

    if val == target:
        print(f'Found {target}!')
        found = True
        break          # breaks inner loop
if found:
    break              # breaks outer loop

```

OUT

Found 5!

## ✦8. COMPREHENSIONS — PYTHONIC ONE-LINERS

Comprehensions create new collections by transforming or filtering an iterable — all in one readable line.

### ► 8a. List Comprehension

#### EXAMPLE

```

# SYNTAX: [expression for item in iterable if condition]

# Traditional loop
squares = []
for x in range(1, 6):
    squares.append(x**2)

# List comprehension – same result, one line
squares = [x**2 for x in range(1, 6)]
print(squares)

# With filter – even squares only
even_sq = [x**2 for x in range(1, 11) if x % 2 == 0]
print(even_sq)

# String manipulation
words = [' hello ', ' world ', ' python ']
clean = [w.strip().upper() for w in words]
print(clean)

```

OUT

```

[1, 4, 9, 16, 25]
[4, 16, 36, 64, 100]
['HELLO', 'WORLD', 'PYTHON']

```

### ► 8b. Dictionary Comprehension

#### EXAMPLE

```

# SYNTAX: {key: value for item in iterable if condition}

# Square of numbers as dict
sq_dict = {x: x**2 for x in range(1, 6)}
print(sq_dict)

# Flip keys and values
original = {'a': 1, 'b': 2, 'c': 3}
flipped = {v: k for k, v in original.items()}

```

```
print(flipped)

# Filter: only items with value > 1
filtered = {k: v for k, v in original.items() if v > 1}
print(filtered)
```

```
OUT      {1:1, 2:4, 3:9, 4:16, 5:25}
         {1:'a', 2:'b', 3:'c'}
         {'b':2, 'c':3}
```

## ► 8c. Set Comprehension

### EXAMPLE

```
# SYNTAX: {expression for item in iterable if condition}

nums = [1, 2, 2, 3, 3, 3, 4]
unique_squares = {x**2 for x in nums} # sets auto-deduplicate
print(unique_squares)
```

```
OUT      {16, 1, 4, 9} # set – unordered, unique
```

## ► 8d. Generator Expression (Lazy Comprehension)

### EXAMPLE

```
# SYNTAX: (expression for item in iterable if condition)
# Use () instead of [] – creates a generator, not a list

gen = (x**2 for x in range(1, 6)) # lazy – computes one at a time
print(next(gen)) # 1
print(next(gen)) # 4

# Great for large data – memory efficient
total = sum(x**2 for x in range(1, 1001)) # no list created!
print(total)
```

```
OUT      1
         4
         333833500
```

## ► 8e. Nested List Comprehension

### ADVANCED

```
# Flatten a 2D matrix
matrix = [[1,2,3],[4,5,6],[7,8,9]]
flat = [val for row in matrix for val in row]
print(flat)

# Transpose a matrix
transposed = [[row[i] for row in matrix] for i in range(3)]
print(transposed)
```

```
OUT      [1, 2, 3, 4, 5, 6, 7, 8, 9]
         [[1,4,7],[2,5,8],[3,6,9]]
```

### ► Comprehension Types — Side by Side

| Syntax                          | Result                                              |
|---------------------------------|-----------------------------------------------------|
| <code>[x for x in lst]</code>   | List comprehension → returns list                   |
| <code>{x:y for x,y in d}</code> | Dict comprehension → returns dict                   |
| <code>{x for x in lst}</code>   | Set comprehension → returns set (unique, unordered) |
| <code>(x for x in lst)</code>   | Generator expression → returns generator (lazy)     |

## 9. COMMON PATTERNS & REAL-WORLD IDIOMS

### ► Accumulator Pattern

#### PATTERN

```
# Sum, product, or collect values
sales = [1200, 3400, 900, 2100, 4500]
total = 0
highest = sales[0]

for sale in sales:
    total += sale
    if sale > highest:
        highest = sale

print(f'Total: {total}, Best: {highest}')
```

OUT

Total: 12100, Best: 4500

### ► Early Exit with break (Search Pattern)

#### PATTERN

```
def find_first(lst, target):
    for i, val in enumerate(lst):
        if val == target:
            return i      # return also acts as break
    return -1            # not found

data = [10, 25, 3, 88, 42, 7]
print(find_first(data, 88)) # 3
print(find_first(data, 99)) # -1
```

OUT

3  
-1

### ► Countdown Timer with while

#### PATTERN

```
import time

def countdown(seconds):
    while seconds > 0:
        print(f'T-{seconds}...')
        time.sleep(1)    # wait 1 second
        seconds -= 1
    print('Blast off! 🚀')

countdown(3)
```

OUT

T-3...  
T-2...  
T-1...  
Blast off! 🚀



## ► FizzBuzz (Classic Interview Question)

### FIZZBUZZ

```
for i in range(1, 21):
    if i % 15 == 0:
        print('FizzBuzz')
    elif i % 3 == 0:
        print('Fizz')
    elif i % 5 == 0:
        print('Buzz')
    else:
        print(i)
```

OUT

```
1
2
Fizz
4
Buzz
Fizz
7
8
Fizz
Buzz
11
Fizz
13
14
FizzBuzz
16
17
Fizz
19
Buzz
```

## 🔗 10. QUICK REFERENCE — ALL CONCEPTS AT A GLANCE

| Concept             | Syntax                            | Key Point                           |
|---------------------|-----------------------------------|-------------------------------------|
| if / elif / else    | if cond: ... elif cond: ... else: | Top-to-bottom, first True wins      |
| Ternary / Inline if | val = a if cond else b            | One-liner simple assignment         |
| Nested if           | if inside if                      | Indent carefully                    |
| Truthy / Falsy      | if value: (no == True needed)     | 0, "", [], None, False are Falsy    |
| match-case          | match x: case 1: ...              | Python 3.10+, like switch           |
| == vs is            | == compares value, is compares id | Never use is for int/str comparison |
| in / not in         | x in list / 'a' in str            | Membership check                    |
| and / or / not      | cond1 and cond2                   | Logical operators                   |
| for loop            | for x in iterable:                | Iterate over any iterable           |
| range()             | range(start, stop, step)          | Generates number sequence           |
| enumerate()         | for i, val in enumerate(lst):     | Index + value together              |
| zip()               | for a, b in zip(lst1, lst2):      | Parallel iteration                  |

|                             |                                          |                                 |
|-----------------------------|------------------------------------------|---------------------------------|
| <b>dict iteration</b>       | <code>.keys() .values() .items()</code>  | Loop over dict parts            |
| <b>while loop</b>           | <code>while condition:</code>            | Repeat until condition False    |
| <b>Infinite loop</b>        | <code>while True: ... break</code>       | Exit with break                 |
| <b>break</b>                | <code>break</code>                       | Exit loop immediately           |
| <b>continue</b>             | <code>continue</code>                    | Skip to next iteration          |
| <b>pass</b>                 | <code>pass</code>                        | Placeholder — do nothing        |
| <b>loop else</b>            | <code>for/while ... else:</code>         | Runs only if no break           |
| <b>Nested loops</b>         | <code>loop inside loop</code>            | Inner runs fully per outer step |
| <b>List comprehension</b>   | <code>[expr for x in lst if cond]</code> | Returns list                    |
| <b>Dict comprehension</b>   | <code>{k:v for k,v in d.items()}</code>  | Returns dict                    |
| <b>Set comprehension</b>    | <code>{expr for x in lst}</code>         | Returns set (unique)            |
| <b>Generator expression</b> | <code>(expr for x in lst)</code>         | Lazy, memory-efficient          |